

# TP & Exercices — Les jointures SQL

## TP A — Parc informatique : Machines / Interventions

### Objectifs

- identifier une clé primaire et une clé étrangère ;
- relier deux tables avec **INNER JOIN** ;
- conserver toutes les lignes d'une table avec **LEFT JOIN** ;
- identifier les absences avec **IS NULL** ;
- réaliser un comptage avec **COUNT** et **GROUP BY**.

### Contexte

Une entreprise souhaite suivre les interventions réalisées sur les machines de son parc informatique.

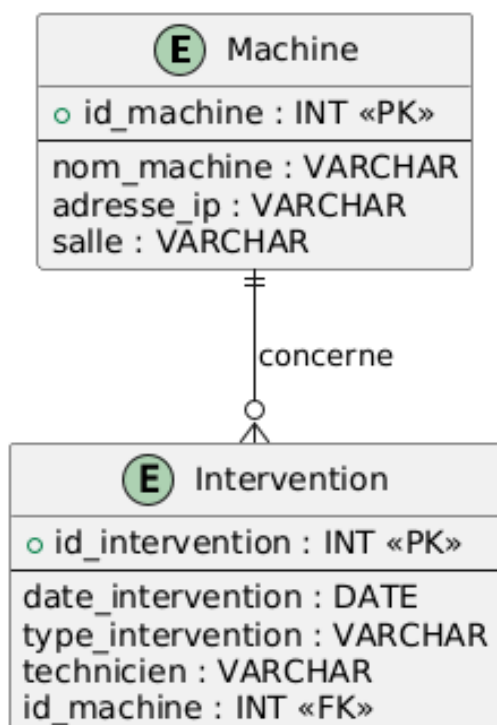
Une machine peut avoir :

- aucune intervention ;
- une intervention ;
- plusieurs interventions.

### Modèle

MACHINE (1) ————— (N) INTERVENTION

### Diagramme UML



- Diagramme UML → identification de la clé étrangère
- Diagramme UML → prédire les résultats des jointures
- Lien direct : « 0 intervention → **LEFT JOIN** obligatoire »

## Structure de la base

### Table **Machine**

| Champ       | Type     |
|-------------|----------|
| id_machine  | INT (PK) |
| nom_machine | VARCHAR  |
| adresse_ip  | VARCHAR  |
| salle       | VARCHAR  |

### Table **Intervention**

| Champ             | Type     |
|-------------------|----------|
| id_intervention   | INT (PK) |
| date_intervention | DATE     |
| type_intervention | VARCHAR  |
| technicien        | VARCHAR  |
| id_machine        | INT (FK) |

## Données

### Table **Machine**

| id_machine | nom_machine | adresse_ip    | salle |
|------------|-------------|---------------|-------|
| 1          | PC-01       | 192.168.10.10 | A101  |
| 2          | PC-02       | 192.168.10.11 | A101  |
| 3          | PC-03       | 192.168.10.12 | B204  |
| 4          | PC-04       | 192.168.10.13 | B204  |
| 5          | PC-05       | 192.168.10.14 | C301  |

### Table **Intervention**

| id_intervention | date_intervention | type_intervention | technicien | id_machine |
|-----------------|-------------------|-------------------|------------|------------|
| 101             | 2024-02-01        | Maintenance       | Alice      | 1          |
| 102             | 2024-02-03        | Panne             | Bob        | 1          |

| id_intervention | date_intervention | type_intervention | technicien | id_machine |
|-----------------|-------------------|-------------------|------------|------------|
| 103             | 2024-02-05        | Maintenance       | Alice      | 3          |
| 104             | 2024-02-06        | Mise à jour       | Claire     | 3          |

**Remarque** : les machines **PC-02**, **PC-04** et **PC-05** n'ont aucune intervention.

Script SQL de création et d'insertion

```
CREATE DATABASE parc_informatique
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;

USE parc_informatique;

CREATE TABLE Machine (
    id_machine INT AUTO_INCREMENT,
    nom_machine VARCHAR(50) NOT NULL,
    adresse_ip VARCHAR(15) NOT NULL,
    salle VARCHAR(10) NOT NULL,
    PRIMARY KEY (id_machine)
);

CREATE TABLE Intervention (
    id_intervention INT AUTO_INCREMENT,
    date_intervention DATE NOT NULL,
    type_intervention VARCHAR(50) NOT NULL,
    technicien VARCHAR(50) NOT NULL,
    id_machine INT NOT NULL,
    PRIMARY KEY (id_intervention),
    FOREIGN KEY (id_machine)
        REFERENCES Machine(id_machine)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

INSERT INTO Machine (id_machine, nom_machine, adresse_ip, salle) VALUES
(1, 'PC-01', '192.168.10.10', 'A101'),
(2, 'PC-02', '192.168.10.11', 'A101'),
(3, 'PC-03', '192.168.10.12', 'B204'),
(4, 'PC-04', '192.168.10.13', 'B204'),
(5, 'PC-05', '192.168.10.14', 'C301');

INSERT INTO Intervention
(id_intervention, date_intervention, type_intervention, technicien,
id_machine)
VALUES
(101, '2024-02-01', 'Maintenance', 'Alice', 1),
(102, '2024-02-03', 'Panne', 'Bob', 1),
(103, '2024-02-05', 'Maintenance', 'Alice', 3),
(104, '2024-02-06', 'Mise à jour', 'Claire', 3);
```

Ces instructions créent d'abord la base, puis les tables ; **Machine** doit être créée avant **Intervention**, car la clé étrangère **id\_machine** référence **Machine.id\_machine**.

## Vérification

Exécuter des requêtes SQL :

```
SELECT * FROM Machine;  
SELECT * FROM Intervention;
```

Vous devez obtenir 5 machines et 4 interventions. Les machines **PC-02**, **PC-04** et **PC-05** n'ont volontairement aucune intervention, ce qui est indispensable pour les exercices sur **LEFT JOIN**, **IS NULL** et **COUNT**.

## Remarques

- **AUTO\_INCREMENT** est adapté aux clés primaires numériques ; MySQL attribue alors automatiquement la valeur suivante lorsque la colonne n'est pas fournie.
- **ON DELETE CASCADE** supprime automatiquement les interventions d'une machine si celle-ci est supprimée.
- **VARCHAR(15)** suffit pour les adresses IPv4.

## Exercices

### Exercice 1 – Comprendre la relation

1. Quelle est la clé primaire de **machine** ?
2. Quelle est la clé primaire de **intervention** ?
3. Quelle colonne permet de relier les tables **Machine** et **Intervention** ?

### Exercice 2 – Machines ayant fait l'objet d'une intervention

Afficher le nom de la machine, la salle, le type d'intervention et le technicien, **uniquement pour les machines ayant eu au moins une intervention**.

**Indice** : seules les correspondances doivent apparaître.

**Résultat attendu : 4 lignes**

### Exercice 3 – Toutes les machines du parc

Afficher le nom de la machine, l'adresse IP et le type d'intervention (s'il existe).

Toutes les machines doivent apparaître, y compris celles sans intervention.

**Résultat attendu : 7 lignes**

Une machine ayant plusieurs interventions apparaît plusieurs fois.

### Exercice 4 – Machines sans intervention

Afficher le nom et la salle des machines qui n'ont jamais fait l'objet d'une intervention.

**Résultat attendu : 3 lignes**

### Exercice 5 – Nombre d'interventions

Afficher pour chaque machine :

```
nom_machine  
nb_interventions
```

Les machines sans intervention doivent apparaître avec 0.

### Résultat attendu : 5 lignes

Corrigés

#### Exercice 1

1. `Machine.id_machine` est la clé primaire de `Machine`.
2. `Intervention.id_intervention` est la clé primaire de `Intervention`.
3. `Intervention.id_machine` est la clé étrangère qui relie les deux tables.

#### Exercice 2 – INNER JOIN

```
SELECT Machine.nom_machine, Machine.salle,  
       Intervention.type_intervention, Intervention.technicien  
FROM Machine  
INNER JOIN Intervention  
ON Machine.id_machine = Intervention.id_machine;
```

On garde uniquement les machines qui ont au moins une intervention.

#### Exercice 3 – LEFT JOIN

```
SELECT Machine.nom_machine, Machine.adresse_ip,  
       Intervention.type_intervention  
FROM Machine  
LEFT JOIN Intervention  
ON Machine.id_machine = Intervention.id_machine;
```

Les machines sans intervention auront `NULL`.

#### Exercice 4 – Machines sans intervention

```
SELECT Machine.nom_machine, Machine.salle  
FROM Machine  
LEFT JOIN Intervention  
ON Machine.id_machine = Intervention.id_machine  
WHERE Intervention.id_intervention IS NULL;
```

`LEFT JOIN + IS NULL` = « ce qui n'existe pas »

## Exercice 5 – Nombre d'interventions par machine

```
SELECT Machine.nom_machine,
       COUNT(Intervention.id_intervention) AS nb_interventions
FROM Machine
LEFT JOIN Intervention
ON Machine.id_machine = Intervention.id_machine
GROUP BY Machine.id_machine, Machine.nom_machine;
```

**COUNT** ignore les **NULL** → machines sans intervention = 0.

## TP B — Parc informatique : Salles / Équipements / Techniciens

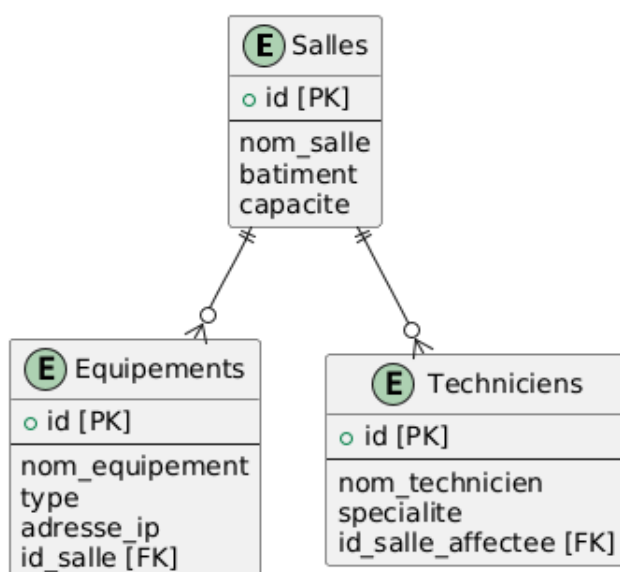
### Objectifs

- comprendre et utiliser les **jointures SQL**,
- choisir le **bon type de JOIN** (**INNER**, **LEFT**),
- distinguer le rôle de **ON** et **WHERE**,
- interpréter les **valeurs NULL**,
- utiliser les **agrégations** (**COUNT**, **AVG**) avec **GROUP BY**.

### Contexte

Vous travaillez pour le service informatique d'un établissement. La base de données permet de gérer les **salles**, les **équipements informatiques** et les **techniciens** responsables. Les informations sont réparties dans plusieurs tables : vous devrez donc utiliser des **jointures**.

### Schéma de la base de données



### Tables et données

#### Table **Salles**

| id | nom_salle | batiment | capacite |
|----|-----------|----------|----------|
| 1  | A101      | A        | 20       |
| 2  | B202      | B        | 15       |
| 3  | C303      | C        | 25       |

Table **Equipements**

| id  | nom_equipement | type       | adresse_ip    | id_salle [FK] |
|-----|----------------|------------|---------------|---------------|
| 101 | PC-Prof        | Ordinateur | 192.168.1.1   | 1             |
| 102 | Switch-Central | Switch     | 192.168.1.254 | 1             |
| 103 | Imprimante     | Imprimante | 192.168.2.1   | 2             |
| 104 | Routeur        | Routeur    | 192.168.1.253 | 3             |

Table **Techniciens**

| id | nom_technicien | specialite | id_salle_affectee [FK] |
|----|----------------|------------|------------------------|
| 1  | Jean Dupont    | Réseau     | 1                      |
| 2  | Marie Martin   | Matériel   | 2                      |
| 3  | Paul Durand    | Sécurité   | NULL                   |

[FK] = clé étrangère, référence une autre table

### Script SQL de création des tables

```
CREATE TABLE Salles (
  id INT PRIMARY KEY,
  nom_salle VARCHAR(50) NOT NULL,
  batiment VARCHAR(10) NOT NULL,
  capacite INT NOT NULL
);

CREATE TABLE Equipements (
  id INT PRIMARY KEY,
  nom_equipement VARCHAR(50) NOT NULL,
  type VARCHAR(50) NOT NULL,
  adresse_ip VARCHAR(15) NOT NULL,
  id_salle INT,
  FOREIGN KEY (id_salle) REFERENCES Salles(id)
);

CREATE TABLE Techniciens (
  id INT PRIMARY KEY,
  nom_technicien VARCHAR(50) NOT NULL,
  specialite VARCHAR(50) NOT NULL,
```

```
id_salle_affectee INT,
FOREIGN KEY (id_salle_affectee) REFERENCES Salles(id)
);
```

## Travail demandé

Pour chaque question : écrivez une requête SQL, respectez les contraintes indiquées, et vérifiez que votre résultat correspond au nombre de lignes attendu.

| Question | Nombre de lignes attendues |
|----------|----------------------------|
| 1        | 4                          |
| 2        | 3                          |
| 3        | 3                          |
| 4        | 2                          |
| 5        | 2                          |
| 6        | 2                          |
| 7        | 3                          |
| 8        | 4                          |
| 9        | 0                          |
| 10       | 2                          |
| 11       | 1                          |
| 12       | 1                          |

## Partie 1 — Jointures simples

**1. Équipements et salles** — Afficher la liste des équipements avec le nom de la salle associée (*nom\_equipement*, *type*, *nom\_salle*). *Indice : tous les équipements sont dans une salle.*

**2. Techniciens et salles (affectation)** — Afficher tous les techniciens avec le nom de leur salle d'affectation, s'ils sont affectés (*nom\_technicien*, *specialite*, *nom\_salle*). Les techniciens sans salle doivent apparaître avec **NULL**. *Indice : certains techniciens ne sont pas affectés.*

**3. Nombre d'équipements par salle** — Afficher chaque salle avec le nombre d'équipements qu'elle contient (*nom\_salle*, *nombre\_equipements*). Les salles sans équipement doivent apparaître.

## Partie 2 — Jointures et filtrage

**4. Filtrer les équipements** — Afficher les équipements de type *Ordinateur* ou *Switch*, avec le nom de la salle (*nom\_equipement*, *type*, *nom\_salle*).

**5. Filtrer les techniciens** — Afficher les techniciens spécialisés en *Réseau* ou *Sécurité*, avec leur salle si affectés (*nom\_technicien*, *specialite*, *nom\_salle*). Les techniciens non affectés doivent apparaître.

**6. Salles du bâtiment A** — Afficher les salles du bâtiment A avec leurs équipements (`nom_salle`, `nom_equipement`, `type`). Une salle sans équipement doit apparaître avec `NULL`.

### Partie 3 — Jointures externes et agrégations

**7. Tous les techniciens** — Afficher tous les techniciens, y compris ceux non affectés à une salle (`nom_technicien`, `specialite`, `nom_salle`).

**8. Équipements par type et bâtiment** — Afficher le nombre d'équipements par type et par bâtiment (`type`, `batiment`, `nombre_equipements`).

**9. Salles sans équipement** — Afficher les salles qui ne contiennent aucun équipement (`nom_salle`).  
(Résultat vide attendu.)

### Partie 4 — Sous-requêtes et jointures avancées

**10. Équipements situés dans une salle affectée à un technicien Réseau** — Afficher les équipements situés dans une salle gérée par un technicien spécialisé en *Réseau* (`nom_equipement`, `type`, `nom_salle`).  
*Indice : plusieurs jointures nécessaires, mais toujours la même logique.*

**11. Salles au-dessus de la capacité moyenne** — Afficher les salles dont la capacité est supérieure à la capacité moyenne (`nom_salle`, `capacite`).

**12. Techniciens dans des salles bien équipées** — Afficher les techniciens affectés à une salle contenant au moins 2 équipements (`nom_technicien`, `nom_salle`).

### Conseils

- Le choix du type de `JOIN` est essentiel.
- Une condition mal placée dans `WHERE` peut annuler un `LEFT JOIN`.
- `NULL` signifie absence de correspondance.
- Vérifiez toujours le nombre de lignes retournées.

### Corrigés

#### 1. Équipements et salles

```
SELECT e.nom_equipement, e.type, s.nom_salle
FROM Equipements e
INNER JOIN Salles s ON e.id_salle = s.id;
```

| nom_equipement | type       | nom_salle |
|----------------|------------|-----------|
| PC-Prof        | Ordinateur | A101      |
| Switch-Central | Switch     | A101      |
| Imprimante     | Imprimante | B202      |
| Routeur        | Routeur    | C303      |

- Chaque équipement possède une `id_salle` ; `INNER JOIN` suffit car un équipement est forcément dans une salle.

## 2. Techniciens et salles d'affectation

```
SELECT t.nom_technicien, t.specialite, s.nom_salle
FROM Techniciens t
LEFT JOIN Salles s ON t.id_salle_affectee = s.id;
```

| nom_technicien | specialite | nom_salle |
|----------------|------------|-----------|
| Jean Dupont    | Réseau     | A101      |
| Marie Martin   | Matériel   | B202      |
| Paul Durand    | Sécurité   | NULL      |

- `LEFT JOIN` permet de conserver tous les techniciens, même sans salle (`NULL`).

## 3. Nombre d'équipements par salle

```
SELECT s.nom_salle, COUNT(e.id) AS nombre_equipements
FROM Salles s
LEFT JOIN Equipements e ON s.id = e.id_salle
GROUP BY s.nom_salle;
-- OU GROUP BY s.id, s.nom_salle
```

| nom_salle | nombre_equipements |
|-----------|--------------------|
| A101      | 2                  |
| B202      | 1                  |
| C303      | 1                  |

- `LEFT JOIN` pour inclure les salles sans équipement ; `COUNT(e.id)` ne compte que les équipements existants.

## 4. Équipements de type Ordinateur ou Switch

```
SELECT e.nom_equipement, e.type, s.nom_salle
FROM Equipements e
INNER JOIN Salles s ON e.id_salle = s.id
WHERE e.type IN ('Ordinateur', 'Switch');
```

| nom_equipement | type       | nom_salle |
|----------------|------------|-----------|
| PC-Prof        | Ordinateur | A101      |

| nom_equipement | type   | nom_salle |
|----------------|--------|-----------|
| Switch-Central | Switch | A101      |

## 5. Techniciens Réseau ou Sécurité

```
SELECT t.nom_technicien, t.specialite, s.nom_salle
FROM Techniciens t
LEFT JOIN Salles s ON t.id_salle_affectee = s.id
WHERE t.specialite IN ('Réseau', 'Sécurité');
```

| nom_technicien | specialite | nom_salle |
|----------------|------------|-----------|
| Jean Dupont    | Réseau     | A101      |
| Paul Durand    | Sécurité   | NULL      |

## 6. Salles du bâtiment A avec équipements

```
SELECT s.nom_salle, e.nom_equipement, e.type
FROM Salles s
LEFT JOIN Equipements e ON s.id = e.id_salle
WHERE s.batiment = 'A';
```

| nom_salle | nom_equipement | type       |
|-----------|----------------|------------|
| A101      | PC-Prof        | Ordinateur |
| A101      | Switch-Central | Switch     |

## 7. Tous les techniciens

```
SELECT t.nom_technicien, t.specialite, s.nom_salle
FROM Techniciens t
LEFT JOIN Salles s ON t.id_salle_affectee = s.id;
```

Même logique que l'exercice 2 : **LEFT JOIN** indispensable pour conserver Paul Durand.

## 8. Équipements par type et bâtiment

```
SELECT e.type, s.batiment, COUNT(e.id) AS nombre_equipements
FROM Equipements e
INNER JOIN Salles s ON e.id_salle = s.id
GROUP BY e.type, s.batiment;
```

| type       | batiment | nombre_equipements |
|------------|----------|--------------------|
| Ordinateur | A        | 1                  |
| Switch     | A        | 1                  |
| Imprimante | B        | 1                  |
| Routeur    | C        | 1                  |

## 9. Salles sans équipement

```
SELECT s.nom_salle
FROM Salles s
LEFT JOIN Equipements e ON s.id = e.id_salle
WHERE e.id IS NULL;
```

**Résultat : aucune ligne retournée** (toutes les salles possèdent au moins un équipement).

## 10. Équipements gérés par un technicien Réseau

```
SELECT e.nom_equipement, e.type, s.nom_salle
FROM Techniciens t
INNER JOIN Salles s ON t.id_salle_affectee = s.id
INNER JOIN Equipements e ON e.id_salle = s.id
WHERE t.specialite = 'Réseau';
```

| nom_equipement | type       | nom_salle |
|----------------|------------|-----------|
| PC-Prof        | Ordinateur | A101      |
| Switch-Central | Switch     | A101      |

- Logique : technicien → salle → équipements. **INNER JOIN** car on cherche des correspondances existantes.

## 11. Salles au-dessus de la capacité moyenne

```
SELECT nom_salle, capacite
FROM Salles
WHERE capacite > (
    SELECT AVG(capacite)
    FROM Salles
);
```

| nom_salle | capacite |
|-----------|----------|
| C303      | 25       |

## 12. Techniciens dans des salles avec $\geq 2$ équipements

```
SELECT t.nom_technicien, s.nom_salle
FROM Techniciens t
INNER JOIN Salles s ON t.id_salle_affectee = s.id
WHERE s.id IN (
    SELECT id_salle
    FROM Equipements
    GROUP BY id_salle
    HAVING COUNT(id) >= 2
);
```

| nom_technicien | nom_salle |
|----------------|-----------|
| Jean Dupont    | A101      |

### Conclusion générale du TP

1. **ON** relie les tables.
2. **WHERE** filtre les résultats.
3. **LEFT JOIN** conserve les lignes sans correspondance.
4. **GROUP BY** est obligatoire avec les fonctions d'agrégation.
5. **NULL** = absence de lien.

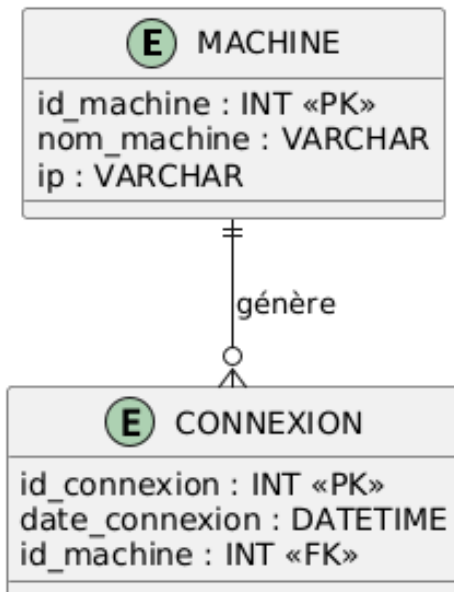
---

## TP C — Réseau : Machines / Connexions

On souhaite analyser les **connexions réseau** dans un parc informatique.

- Une **machine** peut se connecter plusieurs fois
- Une **machine** peut ne jamais s'être connectée
- Une **connexion** peut être orpheline (machine inconnue ou supprimée)

Schéma de la base de données



- Une machine peut générer **0 à n connexions**
- Une connexion peut être **sans machine** (*id\_machine NULL*)

## Données de départ

### MACHINE

| id_machine | nom_machine | ip           |
|------------|-------------|--------------|
| 1          | PC-ADMIN    | 192.168.1.10 |
| 2          | PC-SALLE1   | 192.168.1.20 |
| 3          | PC-SALLE2   | 192.168.1.21 |
| 4          | PC-LABO     | 192.168.1.30 |

### CONNEXION

| id_connexion | date_connexion   | id_machine |
|--------------|------------------|------------|
| 101          | 2024-11-10 08:10 | 1          |
| 102          | 2024-11-10 08:30 | 2          |
| 103          | 2024-11-10 09:00 | 2          |
| 104          | 2024-11-10 09:15 | NULL       |

## SQL – Création de la base et des tables (XAMPP)

```

CREATE DATABASE reseau_bts
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;

USE reseau_bts;
  
```

```
CREATE TABLE MACHINE (  
    id_machine INT AUTO_INCREMENT,  
    nom_machine VARCHAR(50) NOT NULL,  
    ip VARCHAR(15) NOT NULL,  
    PRIMARY KEY (id_machine)  
);  
  
CREATE TABLE CONNEXION (  
    id_connexion INT AUTO_INCREMENT,  
    date_connexion DATETIME NOT NULL,  
    id_machine INT NULL,  
    PRIMARY KEY (id_connexion),  
    FOREIGN KEY (id_machine) REFERENCES MACHINE(id_machine)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);  
  
INSERT INTO MACHINE (nom_machine, ip) VALUES  
( 'PC-ADMIN', '192.168.1.10'),  
( 'PC-SALLE1', '192.168.1.20'),  
( 'PC-SALLE2', '192.168.1.21'),  
( 'PC-LAB0', '192.168.1.30');  
  
INSERT INTO CONNEXION (date_connexion, id_machine) VALUES  
( '2024-11-10 08:10:00', 1),  
( '2024-11-10 08:30:00', 2),  
( '2024-11-10 09:00:00', 2),  
( '2024-11-10 09:15:00', NULL);
```

## Vérification

```
SELECT * FROM MACHINE;  
SELECT * FROM CONNEXION;
```

## Exercices

1. Afficher **toutes les machines** avec leurs dates de connexion (s'il y en a). **Résultat attendu : 5 lignes**
2. Afficher **les machines qui ne possèdent aucune connexion enregistrée**. **Résultat attendu : 2 lignes**
3. Afficher **les connexions pour lesquelles aucune machine n'est associée**. **Résultat attendu : 1 ligne**
4. Afficher le **nombre de connexions par machine**, y compris celles qui ont 0 connexion. **Résultat attendu : 4 lignes**
5. Afficher les **machines ayant réalisé au moins deux connexions**. **Résultat attendu : 1 ligne**

Préciser le type de jointure utilisé pour chaque question.

## Corrigés

### 1. Toutes les machines + connexions

```
SELECT nom_machine, ip, date_connexion
FROM MACHINE
LEFT JOIN CONNEXION
ON MACHINE.id_machine = CONNEXION.id_machine;
```

| nom_machine | ip           | date_connexion   |
|-------------|--------------|------------------|
| PC-ADMIN    | 192.168.1.10 | 2024-11-10 08:10 |
| PC-SALLE1   | 192.168.1.20 | 2024-11-10 08:30 |
| PC-SALLE1   | 192.168.1.20 | 2024-11-10 09:00 |
| PC-SALLE2   | 192.168.1.21 | NULL             |
| PC-LABO     | 192.168.1.30 | NULL             |

**LEFT JOIN** = je garde toutes les machines

### 2. Machines jamais connectées

```
SELECT nom_machine, ip
FROM MACHINE
LEFT JOIN CONNEXION
ON MACHINE.id_machine = CONNEXION.id_machine
WHERE CONNEXION.id_connexion IS NULL;
```

| nom_machine | ip           |
|-------------|--------------|
| PC-SALLE2   | 192.168.1.21 |
| PC-LABO     | 192.168.1.30 |

**NULL** = aucune correspondance trouvée.

### 3. Connexions orphelines

```
SELECT id_connexion, date_connexion
FROM CONNEXION
LEFT JOIN MACHINE
ON CONNEXION.id_machine = MACHINE.id_machine
WHERE MACHINE.id_machine IS NULL;
```

OU

```
SELECT c.id_connexion, c.date_connexion
FROM CONNEXION c
LEFT JOIN MACHINE m
  ON c.id_machine = m.id_machine
WHERE m.id_machine IS NULL;
```

| id_connexion | date_connexion   |
|--------------|------------------|
| 104          | 2024-11-10 09:15 |

#### 4. Nombre de connexions par machine

```
SELECT nom_machine, COUNT(CONNEXION.id_connexion) AS nb_connexions
FROM MACHINE
LEFT JOIN CONNEXION
  ON MACHINE.id_machine = CONNEXION.id_machine
GROUP BY nom_machine;
```

| nom_machine | nb_connexions |
|-------------|---------------|
| PC-ADMIN    | 1             |
| PC-SALLE1   | 2             |
| PC-SALLE2   | 0             |
| PC-LABO     | 0             |

`COUNT(colonne)` ignore les `NULL`.

#### 5. Machines ayant réalisé au moins deux connexions

```
SELECT MACHINE.nom_machine,
       COUNT(CONNEXION.id_connexion) AS nb_connexions
FROM MACHINE
LEFT JOIN CONNEXION
  ON MACHINE.id_machine = CONNEXION.id_machine
GROUP BY MACHINE.id_machine, MACHINE.nom_machine
HAVING COUNT(CONNEXION.id_connexion) >= 2;
```

| nom_machine | nb_connexions |
|-------------|---------------|
| PC-SALLE1   | 2             |

`HAVING` filtre après le `GROUP BY` ; il est nécessaire car la condition porte sur le résultat de `COUNT`.

# TP final — Médiathèque

## Objectif

Consolider toutes les compétences :

- utiliser des jointures **INNER JOIN** et **LEFT JOIN** ;
- filtrer les résultats avec **WHERE** et **IS NULL** ;
- utiliser des fonctions d'agrégation comme **COUNT()** et **MAX()** ;
- regrouper les résultats avec **GROUP BY** ;
- effectuer des requêtes impliquant plusieurs tables ;
- comprendre et exploiter des relations **1–N** entre les tables.

## Contexte

Vous travaillez pour une **médiathèque municipale**.

La base de données permet de gérer :

- les **adhérents** de la médiathèque ;
- les **livres** disponibles ;
- les différents **exemplaires** de chaque livre ;
- les **prêts** effectués par les adhérents.

Un même adhérent peut effectuer plusieurs prêts et un même livre peut posséder plusieurs exemplaires.

## Schéma de la base de données

ADHERENTS (1) — (N) PRETS (N) — (1) EXEMPLAIRES (N) — (1)  
LIVRES

## Tables

### ADHERENTS

- **id** : identifiant de l'adhérent (**PK**)
- **nom**
- **prenom**
- **email**

### LIVRES

- **id** : identifiant du livre (**PK**)
- **titre**
- **auteur**
- **genre**

### EXEMPLAIRES

- **id** : identifiant de l'exemplaire (**PK**)

- **id\_livre** : identifiant du livre (**FK**)
- **etat** : état de l'exemplaire (**neuf**, **bon** ou **abîmé**)

## PRETS

- **id** : identifiant du prêt (**PK**)
- **id\_adherent** : identifiant de l'adhérent (**FK**)
- **id\_exemplaire** : identifiant de l'exemplaire (**FK**)
- **date\_pret**
- **date\_retour\_prevue**
- **date\_retour\_effective** : **NULL** si le livre n'a pas encore été rendu

## Données

### adherents

| id | nom     | prenom | email                                            |
|----|---------|--------|--------------------------------------------------|
| 1  | Martin  | Alice  | <a href="mailto:alice@mail.fr">alice@mail.fr</a> |
| 2  | Dupont  | Lucas  | <a href="mailto:lucas@mail.fr">lucas@mail.fr</a> |
| 3  | Bernard | Emma   | <a href="mailto:emma@mail.fr">emma@mail.fr</a>   |
| 4  | Petit   | Hugo   | <a href="mailto:hugo@mail.fr">hugo@mail.fr</a>   |

### livres

| id | titre           | auteur        | genre     |
|----|-----------------|---------------|-----------|
| 1  | Les Misérables  | Victor Hugo   | Roman     |
| 2  | 1984            | George Orwell | Science-F |
| 3  | Le Petit Prince | Saint-Exupéry | Conte     |
| 4  | Dune            | Frank Herbert | Science-F |

### exemplaires

| id  | id_livre | etat  |
|-----|----------|-------|
| 101 | 1        | bon   |
| 102 | 1        | abîmé |
| 103 | 2        | neuf  |
| 104 | 3        | bon   |
| 105 | 4        | neuf  |

### prets

| id | id_adherent | id_exemplaire | date_pret  | date_retour_prevue | date_retour_effective |
|----|-------------|---------------|------------|--------------------|-----------------------|
| 1  | 1           | 101           | 2024-09-01 | 2024-09-15         | 2024-09-14            |
| 2  | 1           | 103           | 2024-09-10 | 2024-09-24         | NULL                  |
| 3  | 2           | 104           | 2024-09-05 | 2024-09-19         | 2024-09-18            |
| 4  | 2           | 105           | 2024-09-12 | 2024-09-26         | NULL                  |
| 5  | 3           | 102           | 2024-09-08 | 2024-09-22         | NULL                  |

**Hugo n'a effectué aucun prêt.**

---

## Travail demandé

### 1. Livres et exemplaires

Afficher le **titre du livre** et l'**état de chacun de ses exemplaires**.

**5 lignes attendues.**

---

### 2. Adhérents et prêts

Afficher le **nom de l'adhérent** et la **date de chacun de ses prêts**.

**5 lignes attendues.**

---

### 3. Tous les adhérents

Afficher **tous les adhérents**, avec la **date de prêt lorsqu'elle existe**.

Les adhérents n'ayant effectué aucun prêt doivent également apparaître.

**6 lignes attendues.**

---

### 4. Adhérents sans prêt

Afficher les adhérents **n'ayant jamais effectué de prêt**.

**1 ligne attendue.**

---

### 5. Prêts actuellement en cours

Afficher :

```
nom  
titre  
date_pret
```

pour les livres qui **n'ont pas encore été rendus**.

**3 lignes attendues.**

---

## 6. Nombre de prêts par adhérent

Afficher :

```
nom  
nombre_prets
```

Tous les adhérents doivent apparaître, **y compris ceux qui n'ont jamais emprunté**.

**4 lignes attendues.**

---

## 7. Synthèse par adhérent

Afficher :

```
nom  
nombre_prets  
dernier_pret
```

pour **tous les adhérents**.

**4 lignes attendues.**

Cette question constitue le **défi principal du TP**.

Elle nécessite notamment l'utilisation de :

```
COUNT()  
MAX()  
GROUP BY  
LEFT JOIN
```

---